SAPPhiREIQ

FULL STACK DEVELOPMENT • STUDENT PROJECT

RealEstate Platform

React + TypeScript + Node.js + Express + MongoDB

A step-by-step project for building a real estate platform with property listings, filters, viewing bookings and user dashboards, plus an optional AI-powered property search as a bonus. Core: 4 weeks. Optional AI: +1 week.

4+1	React TS	Express	AI
Core + Optional	Frontend	Backend	Optional Add-on



React • Node • Mongo

SAPPhiREIQ

01 / PROJECT OVERVIEW

RealEstate Platform

A real estate website with listings and booking, and an optional AI search add-on.

Project Description

Students build a real estate platform for India with a user website and a REST API. Users browse and filter properties, list their own, and book viewings. As an optional bonus, an AI Property Hub can search live listings from property portals and rank the best matches.

Project Scope

Core: JWT authentication with password reset by email, property CRUD with multi-image upload, filters and gallery, guest and logged-in booking, a user dashboard, security and deployment. Optional: the AI search pipeline, kept behind a feature switch.

Prerequisites

- **React and TypeScript:** Components, hooks, props and types.
- **Express and MongoDB:** REST APIs, Mongoose and JWT auth.
- **Async JavaScript:** Promises, async/await and error handling.
- **Accounts:** MongoDB Atlas, ImageKit and an email (SMTP) service. Only for the optional AI module: a Firecrawl free key.

Target audience: Intermediate to advanced students who have finished at least one full stack project with authentication.

SAPPhiREIQ

02 / PROJECT ROADMAP

RealEstate Platform Roadmap (4 Weeks + Optional AI)

Weeks 1-4: modules 1-4 (core). Week 5: module 5 (optional AI bonus).

#	MODULE	CORE FOCUS	KEY TECHNOLOGIES
1	**Project Setup, Models & Authentication**	Foundation	Express, Mongoose, JWT, bcrypt, Nodemailer
2	**Properties, Filters & Image Upload**	Core Feature	Multer, ImageKit, React Router, Tailwind
3	**Appointments & User Dashboard**	User Flow	Appointment API, AuthContext, protected routes
4	**Security, SEO & Deployment**	Production Readiness	Helmet, CORS, rate limiting, Vercel, Render
5	**Optional Bonus: AI Property Hub**	Optional Add-on	Firecrawl, AI model, deduplication, retries

Student Workflow

Finish and deploy the four core modules first. Take on the AI module only if time allows, and keep it switchable so the platform works without it.

SAPPhiREIQ

02 / MODULE 1

Project Setup, Models & Authentication

Build the RealEstate platform step by step using React, TypeScript, Node.js, Express and MongoDB.

Learning Objectives

- Set up two folders: frontend and backend.
- Model Property, User and Appointment.
- Build register, login and password reset.

Core Concepts

- Two-App Layout: a React frontend and one Express API.
- Password Safety: bcrypt hashing and reset tokens sent by email.
- Environment Config: secrets kept in .env files.

Tools & Technologies

- Express
- Mongoose
- JWT
- bcrypt
- Nodemailer

Hands-on Student Exercises

Register a user, log in, request a password reset email, and fetch the current user from GET /api/users/me with the token.

SAPPhiREIQ

03 / MODULE 2

Properties, Filters & Image Upload

Build the RealEstate platform step by step using React, TypeScript, Node.js, Express and MongoDB.

Learning Objectives

- Let logged-in users create, edit and delete their own properties.
- Upload up to four images per property to a CDN.
- Build the listing page with filters and a gallery.

Core Concepts

- Ownership: users manage only their own listings.
- Multipart Upload: Multer receives files and ImageKit stores them.
- Filter Query: price, type, location and area as query values.
- Lazy Loaded Pages: React.lazy for faster loading.

Tools & Technologies

- Multer
- ImageKit
- React Router
- Tailwind CSS

Hands-on Student Exercises

Add a property with four images, filter the list by price and type, and confirm a second user cannot edit it.

SAPPhiREIQ

04 / MODULE 3

Appointments & User Dashboard

Build the RealEstate platform step by step using React, TypeScript, Node.js, Express and MongoDB.

Learning Objectives

- Let guests and logged-in users book a viewing.
- Let users cancel their appointments.
- Build a dashboard with my listings and my appointments.

Core Concepts

- Guest vs Auth Booking: two routes, one model.
- Appointment Status: pending, confirmed or cancelled.
- Protected Routes: dashboard only for logged-in users.

Tools & Technologies

- Express Router
- React Context
- Axios

Hands-on Student Exercises

Book a viewing as a guest, then log in, add a listing, and see it and your appointments on the dashboard.

SAPPhiREIQ

05 / MODULE 4

Security, SEO & Deployment

Build the RealEstate platform step by step using React, TypeScript, Node.js, Express and MongoDB.

Learning Objectives

- Add Helmet, CORS rules and a rate limiter.
- Add SEO tags, sitemap and robots.txt.
- Deploy the frontend and the API.

Core Concepts

- CORS Allow List: only your own frontend URL.
- Environment Variables: separate local and production values.
- Deployment: frontend on Vercel, API on Render.

Tools & Technologies

- Helmet
- express-rate-limit
- Vercel
- Render
- MongoDB Atlas

Hands-on Student Exercises

Deploy both apps, set the environment variables, and test a deep link such as /dashboard on the live site.

SAPPhiREIQ

06 / MODULE 5

Optional Bonus: AI Property Hub

Build the RealEstate platform step by step using React, TypeScript, Node.js, Express and MongoDB.

Learning Objectives

- Add the AI search only after the core platform works.
- Scrape individual listing pages and rank results with an AI model.
- Keep the feature switchable so the app works without it.

Core Concepts

- Search to Scrape: find listing URLs first, then scrape each page.
- Deduplicate and Filter: remove repeats and rentals in code.
- Bring Your Own Key: the user's scraping key is sent in a request header.
- Retry with Backoff: handle rate limits and server errors.
- Feature Flag: a setting such as VITE_ENABLE_AI_HUB turns the AI feature on or off.

Tools & Technologies

- Firecrawl
- AI model API
- Axios
- localStorage

Hands-on Student Exercises

Optional: Search for a 2BHK flat in a city, view ranked results with source badges, and trigger a rate-limit error to see the retry work. Then switch the feature off and confirm the rest of the app still works.

07 / PROJECT OUTCOME

What You Will Build

The completed core project is a deployed real estate website with listings and booking. The AI search is an optional add-on.

Application Structure

React Website

Listings, booking, dashboard

Express API

Auth, properties, appointments

MongoDB

Users, properties, appointments

The React app calls the Express API, which uses MongoDB for data and ImageKit for images. AI services are used only if the optional module is enabled.

Skills Practiced

- Connecting a React TypeScript frontend to a custom API.
- JWT auth and password reset by email.
- Multi-image upload with a CDN.
- Filtering, ownership rules and booking flows.
- Securing and deploying a production application.
- Optional: external APIs with retries, AI ranking and feature flags.

Use the module exercises as checkpoints while building your application.